

## Three ways to return a variable:

## joinSpheres-returnByValue.cpp

```

15 Sphere joinSpheres(const Sphere &s1, const Sphere &s2) {
16     double totalVolume = s1.getVolume() + s2.getVolume();
17
18     double newRadius = std::pow(
19         (3.0 * totalVolume) / (4.0 * 3.141592654),
20         1.0/3.0
21     );
22
23     return Sphere(newRadius);
24 }

```

## joinSpheres-returnByPointer.cpp

```

15 Sphere *joinSpheres(const Sphere &s1, const Sphere &s2) {
16     double totalVolume = s1.getVolume() + s2.getVolume();
17
18     double newRadius = std::pow(
19         (3.0 * totalVolume) / (4.0 * 3.141592654),
20         1.0/3.0
21     );
22
23     return new Sphere(newRadius);
24 }

```

## joinSpheres-returnByReference.cpp

```

15 Sphere & joinSpheres(const Sphere &s1, const Sphere &s2) {
16     double totalVolume = s1.getVolume() + s2.getVolume();
17
18     double newRadius = std::pow(
19         (3.0 * totalVolume) / (4.0 * 3.141592654),
20         1.0/3.0
21     );
22     //
23     // BAD PRATICE: _____
24     //
25     Sphere *result = new Sphere(newRadius);
26     return *result; // Memory must be delete'd later
27 }
28
29 //
30 // ERROR: _____
31 //
32 Sphere result(newRadius);
33 return result;
34 }

```

Return by reference takeaway:

## Adding our Sphere class:

What should happen when we add two **Sphere** classes?

## addSpheres.cpp

```

1 #include "sphere.h"
2
3 int main() {
4     cs225::Sphere s1(3), s2(4);
5     cs225::Sphere s3 = s1 + s2;
6     return 0;
7 }

```

## Overloading Operators

C++ allows custom behaviors to be defined on over 20 operators:

| Arithmetic | + - * / % ++ -- |
|------------|-----------------|
| Bitwise    | &   ^ ~ << >>   |
| Assignment | =               |
| Comparison | == != > < >= <= |
| Logical    | ! &&            |
| Other      | [] () ->        |

General Syntax:

Adding overloaded operators to Sphere:

| sphere.h           | sphere.cpp    |
|--------------------|---------------|
| 1 #ifndef SPHERE_H | ... /* ... */ |
| 2 #define SPHERE_H | 10            |
| 3                  | 11            |
| 4 class Sphere {   | 12            |
| 5 public:          | 13            |
| ... // ...         | 14            |
| 17                 | 15            |
| 18                 | 16            |
| 19                 | 17            |
| 20                 | 18            |
| 21                 | 19            |
| ... // ...         | 20            |
| 30 private:        | 21            |
| 31 double r_;      | 22            |
| 32 };              | 23            |
| 33                 | 24            |
| 34 #endif          | ... /* ... */ |

## One Very Special Operator: \_\_\_\_\_

1. Similar to the Copy Constructor:
2. Different than the Copy Constructor:

| sphere.h |                  | sphere.cpp |           |
|----------|------------------|------------|-----------|
| 1        | #ifndef SPHERE_H | ...        | /* ... */ |
| 2        | #define SPHERE_H | 10         |           |
| 3        |                  | 11         |           |
| 4        | class Sphere {   | 12         |           |
| 5        | public:          | 13         |           |
| ...      | // ...           | 14         |           |
| 17       |                  | 15         |           |
| 18       |                  | 16         |           |
| 19       |                  | 17         |           |
| 20       |                  | 18         |           |
| 21       |                  | 19         |           |
| ...      | // ...           | 20         |           |
| 30       | private:         | 21         |           |
| 31       | double r_;       | 22         |           |
| 32       | };               | 23         |           |
| 33       |                  | 24         |           |
| 34       | #endif           | ...        | /* ... */ |

### Big Idea: Destructor

| sphere.h |                  | sphere.cpp |           |
|----------|------------------|------------|-----------|
| 1        | #ifndef SPHERE_H | ...        | /* ... */ |
| 2        | #define SPHERE_H | 10         |           |
| 3        |                  | 11         |           |
| 4        | class Sphere {   | 12         |           |
| 5        | public:          | 13         |           |
| ...      | // ...           | 14         |           |
| 17       |                  | 15         |           |
| 18       |                  | 16         |           |
| ...      | // ...           | ...        | /* ... */ |

## The “Rule of Three”:

- 1.
- 2.
- 3.

Why are the big three so important?

Example:

| sphere.h |                  | sphere.cpp |           |
|----------|------------------|------------|-----------|
| 1        | #ifndef SPHERE_H | ...        | /* ... */ |
| 2        | #define SPHERE_H | 10         |           |
| 3        |                  | 11         |           |
| 4        | class Sphere {   | 12         |           |
| 5        | public:          | 13         |           |
| ...      | // ...           | 14         |           |
| 17       |                  | 15         |           |
| 18       |                  | 16         |           |
| ...      | // ...           | ...        | /* ... */ |

## CS 225 – Things To Be Doing:

1. How was your quiz?
2. MP1 due on Monday
3. MP2 released after MP1 is due (*start early for +EC!*)
4. Happy Winter break!